

Pre-lab Exercises

Exercise 1:

At the moment, in `geometric_IK.py`, `geometric_inv_kin()` returns only `allPhi`. Edit `geometric_inv_kin()` so that we can pass in an array of joint positions as an argument (`phi_prev`), then the closest valid solution is outputted as `bestPhi`.

Description	Position (x, y, z)	wrist	phi_prev	allPhi	bestPhi
Normal	[0.3, 0.4, 0.35]	0.87	[0, 0, 0, 0]	$\begin{pmatrix} -2.21 & -2.21 & 0.93 & 0.93 \\ -2.14 & -0.49 & 0.2 & 1.86 \\ 0.11 & -2.97 & 0.11 & -2.97 \\ 0.87 & 0.87 & 0.87 & 0.87 \end{pmatrix}$	$\begin{pmatrix} 0.93 \\ 0.2 \\ 0.11 \\ 0.87 \end{pmatrix}$
Invalid wrist	[0.3, 0.4, 0.35]	3.49	[0, 0, 0, 0]	$\begin{pmatrix} -2.21 & -2.21 & 0.93 & 0.93 \\ -2.14 & -0.49 & 0.2 & 1.86 \\ 0.11 & -2.97 & 0.11 & -2.97 \\ -2.79 & -2.79 & -2.79 & -2.79 \end{pmatrix}$	Fail
Negative Position	[-0.2, -0.2, 0.6]	-2.62	[0, 0, 0, 0]	$\begin{pmatrix} 0.79 & 0.79 & -2.36 & -2.36 \\ -1.53 & 0.14 & -0.42 & 1.24 \\ 0.12 & -2.98 & 0.12 & -2.98 \\ -2.62 & -2.62 & -2.62 & -2.62 \end{pmatrix}$	$\begin{pmatrix} -2.36 \\ -0.42 \\ 0.12 \\ -2.62 \end{pmatrix}$
Out of workspace	[0.75, 0.3, 0.5]	0	[0, 0, 0, 0]	Fail	Fail
Singularity	[0, 0, 0.75]	0	[0, 0, 0, 0]	$\begin{pmatrix} -3.14 & 0. & -3.14 & 0. \\ -0.82 & 0.53 & -0.82 & 0.53 \\ -0.17 & -2.69 & -0.17 & -2.69 \\ 0. & 0. & 0. & 0. \end{pmatrix}$	Fail
Singularity in between	[0, -0.45, 0.5]	0	[0, 0, 0, 0]	$\begin{pmatrix} 1.57 & 1.57 & -1.57 & -1.57 \\ -1.79 & -0.28 & 0.00 & 1.51 \\ -0.03 & -2.83 & -0.03 & -2.83 \\ 0.00 & 0.00 & 0.00 & 0.00 \end{pmatrix}$	$\begin{pmatrix} -1.57 \\ 0 \\ -0.03 \\ 0 \end{pmatrix}$

Note: How you represent the fail condition is up to you, the recommendation is to return `phi_prev`.

Exercise 2: In `numerical_IK.py`, implement a numerical solver for the QArm using the provided `Inv_Jacobian()` function to get the inverse Jacobian. Several useful helper functions are listed in the python file as comments. Use the same test cases as before.

Description	Position (x, y, z)	wrist	phi_prev	bestPhi
Normal	[0.3, 0.4, 0.35]	0.87	[0, 0, 0, 0]	$\begin{pmatrix} 0.93 \\ 0.2 \\ 0.11 \\ 0.87 \end{pmatrix}$
Invalid wrist	[0.3, 0.4, 0.35]	3.49	[0, 0, 0, 0]	Fail
Negative Position	[-0.2, -0.2, 0.6]	-2.62	[0, 0, 0, 0]	$\begin{pmatrix} -2.36 \\ -0.42 \\ 0.12 \\ -2.62 \end{pmatrix}$
Out of workspace	[0.75, 0.3, 0.5]	0	[0, 0, 0, 0]	Fail
Singularity	[0, 0, 0.75]	0	[0, 0, 0, 0]	Fail
Singularity in between	[0, -0.45, 0.5]	0	[0, 0, 0, 0]	Fail

Note: Since the numerical solver stops at a threshold, your values will be slightly different than the exact values provided by the geometric approach.

Note: How you represent the fail condition is up to you, the recommendation is to return `phi_prev`.

Exercise 3:

Answer the following conceptual questions on Inverse Kinematics:

1. Why does the analytical geometric approach produce up to four distinct solutions for a given (x, y, z) position, and why is `phi_prev` necessary to select the `bestPhi` for continuous robot motion?
2. In your numerical test cases, why did the solver fail to converge at $[0, 0, 0.75]$ and $[0, -0.45, 0.5]$? What happens to the Jacobian inverse $\mathbf{J}^{-1}$ near kinematic singularities?

In-Lab Exercises

Exercise 4: Create an RRT algorithm in the file `obstacle_avoidance_RRT.py`.

Note: Be careful with step sizes, make it too large and you may “skip” through an obstacle.

Test and demonstrate your program across both obstacle setups. When verified, show your working demonstration to the TA.

Exercise 5: In `obstacle_avoidance_potential.py`, implement obstacle avoidance using artificial potential fields. All variables have been assigned a starting value that will be roughly correct; once implemented, do some tuning to improve performance.

Test and demonstrate your program across both obstacle setups. When verified, show your working demonstration to the TA.

Post-lab Exercises

Exercise 6:

Answer the following comparative questions for **Inverse Kinematics**:

1. Compare the geometric and numerical IK solvers. What are their respective pros and cons? Consider:
 - Did one solver encounter issues or failures where the other did not?
 - In what practical robotics scenarios would you choose an analytical solver over a numerical solver (and vice versa)?
 - How consistent are the solvers in terms of computation runtime and final positioning accuracy?
2. What modifications could improve each method? (For example, how does a Damped Least Squares / Damped Inverse $\dot{\underline{q}} = (\underline{J} + \lambda \underline{I})^{-1} \underline{v}$ mitigate numerical divergence near singularities?)

Exercise 7:

Answer the following comparative questions for **Path Planning**:

1. Now that you have implemented both RRT and Artificial Potential Fields, compare their performance and characteristics:
 - How do their planning/execution speeds compare?
 - Can the manipulator get “stuck”? Why? (Explain the local minima trap in potential fields vs. probabilistic completeness in RRT).
 - Assuming you had an accurate map of the environment, how would each method handle an environment with moving or dynamic obstacles?
 - How do they scale to more complex obstacles or higher-DOF manipulators?
2. What improvements could you make to each method? What are some additions or changes that you can make to address the shortcomings of each method?